Homework 4

CS Osvita

February 2, 2026

Homework 4 is a Bit about Bytes. It is designed to increase your familiarity with how we encode and decode values to and from their binary representations. The more comfortable you are with these encodings, the more you'll notice that very little can be done with straight ASCII. For instance, most protocols for data interchange — such as network protocols and file formats — are binary formats.

1 Protobuf

In this exercise, you will implement the Base 128 Varint encoding used in Protocol Buffers. This is a good reason to play with fun data encoding while exposing you to basic concepts in working with binary data, such as reading hexadecimal, byte ordering, and bitwise operations.

You should write an encode function that takes an unsigned 64-bit integer and returns a sequence of bytes in the Varint encoding that Protocol Buffers uses. You should also write a decode function that does the inverse.

Some example inputs are provided in the exercise files as unsigned 64-bit integers in case it's hard to construct these as literals in your language of choice (you are welcome to use any). A simple test of correctness in Python might look like:

```
assert encode(150) == b'\x96\x01'
assert decode(b'\x96\x01') == 150
```

You could also roundtrip test randomly or a range of inputs, e.g., to test the first 1 billion inputs:

```
for n in range(1 << 30):
    assert decode(encode(n)) == n
```

2 Twitter

Imagine you're a social network with a 280-byte limit for certain text. Since some characters require multiple bytes to encode, blindly truncating a string

after 280 bytes could leave junk bytes trailing. This problem is both a good reason to understand how the UTF-8 encoding works and representative of a common issue in practice.

The "cases" file has one case per line. The first byte is an unsigned integer indicating the number of bytes to which to truncate; the remaining is a UTF-8 encoded string to truncate. In other words, the first byte indicates the maximum bytes to keep. Then, the remaining bytes on the line form a single UTF-8 string. There is one length byte and one string, per line. The "expected" file has one expected result per line.

Your program should read "cases", parse them, and write one correctly truncated string per line. This output can then be diffed against "expected", so you can test your solution like this:

```
diff expected <(python3 twitter.py cases)
```

diff is a utility used to compare files. It shows the difference between two files line by line. Of course, the point of the exercise is to avoid truncating the string in the middle of a single Unicode codepoint and avoid using code like str.decode("utf-8").

3 CSS

In CSS, two ways to specify colors are hexadecimal and RGB. For instance, pure red would be 0xff0000 or RGB(255, 0, 0). In this problem, you will write a program to convert color declarations in CSS files from hexadecimal to RGB form.

The primary objective is to give you more exposure to hexadecimal as an output format for binary data, in a realistic context. You should try to solve the problem by doing the hexadecimal to decimal conversion with minimal use of library functions.

Test files are provided. You may want to start with more discrete tests like:

```
assert hex_to_rgb('#00ff00') == 'rgb(0, 255, 0)'
```

Once you've implemented your solution, you can verify its correctness by running:

```
diff <(cat advanced.css | python3 css.py) advanced_expected.css
```

If the **diff** command produces no output, it means your solution is correct, as there are no differences between the generated output and the expected results.

4 Byte ordering

When we write the decimal number 123, we all know to interpret the 1 as the highest-order digit, and the 3 as the lowest. This is arbitrary: we could have

decided to write it as 321 instead, and so long as we were all in agreement, this would have been fine.

We don't typically need to worry about the order of bits in a binary number since the smallest unit of data we retrieve from RAM is an entire byte, and the hardware maintains consistent bit ordering for us.

However, if we have a multi-byte value, we must decide whether the highest-order bytes should come first or last. For instance, the number 256 could be written either 0b00000001 00000000 or 0b00000000 00000001 (spaces inserted for clarity). One may seem more "obvious" to you than the other, but we could use either, and in fact, software engineers are forced to use both, due to our classic inability to agree on anything. The names we give to the two-byte orderings are *big-endian* and *little-endian*.

Big-endian ordering places the highest value byte first (to the left). This is the scheme typically used for decimal numbers like 123, as well as the typical encoding for network protocols. Little-endian ordering places the lowest value byte on the left, which is a good way to remember the difference ("Lowest on the Left in Little-endian"). Intel machines use little-endian encoding.

For the following exercise, you may find it helpful to refer to CS:APP chapter 2.1.3 and/or the video Byte ordering.

4.1 It's over 9000!

We've encoded the number 9001 in the file 9001. Did we use big-endian or little-endian encoding? You may wish to use xxd in either the default (hex) mode or with the -b flag for binary. No coding required for this exercise—just use pen and paper.

4.2 TCP (Optional)

When sending data across networks, computers need to be able to parse data and agree upon its meaning. TCP is responsible for port-to-port delivery of "segments' of a message, each of which needs to be acknowledged, so we would expect to see fields such as the source and destination port and sequence and acknowledgment number in the TCP header.

In TCP — like most network protocols — the byte ordering is big-endian. This is specified in the protocol rather than encoding in the header to simplify parsing and save a little space. Consider that if we didn't have an agreed-upon byte ordering (or communicate the byte ordering in some way), it would be very hard for both sides to understand one another!

The file *tcpheader* contains the first 16 bytes extracted from a TCP header. Using a diagram of the TCP segment structure, can you determine the values of the sequence number, acknowledgment number, source port and destination port? **Hint**: Looking at the header specification, we can see that sequence number, acknowledgment number, source and destination port are available at offsets 4, 8, 0 and 2 and have lengths 4, 4, 2 and 2 bytes respectively.

As a stretch goal, see if you can determine the specified length of the TCP header in *tcpheader* (remember, we have shared only the first 16 bytes). Looking at the header specification again, can you determine if ours contained any optional fields? **Hint**: The data offset field is what helps us determine the header length. It is the high order 4 bits of the 12 byte, and encodes the number of 32 bit words of header, so we must multiply by 4 to get our answer.

5 IEEE Floating Point

For this exercise, there's no need to write any code. Simply use pen and paper to work through it. You may wish to read chapter 2.4 of CS:APP and/or watch the video IEEE Floating Point.

Identify the 3 components of this 32-bit IEEE Floating Point Number and their values:

```
01000010001010100000000000000000
```

1. Using the three components, compute the value this represents.
2. For the largest fixed exponent, 11111110 = 254 - 127 = 127, what is the smallest (magnitude) incremental change that can be made to a number?
3. For the smallest (most negative) fixed exponent, what is the smallest (magnitude) incremental change that can be made to a number?

Further Resources

For those following CS:APP, we suggest Chapter 2: "Representing and Manipulating Information".

What Every Computer Scientist Should Know About Floating-Point Arithmetic is a classic and stunningly detailed paper covering the many rough edges and surprising behaviors of the standard.

Joel Spolsky on Unicode and character sets is another great resource for understanding character encoding from a programmer's perspective.

You may wish to watch Why We Use Two's Complement.